

PROYECTO

CONTEXTO

En la actualidad, muchas de las interacciones entre jóvenes y adultos se dan en el contexto de comunidades digitales, en ellas, personas de diversas partes del mundo se unen a comunidades en función de sus gustos, intereses e intenciones. Dentro de las plataformas para el desarrollo de comunidades digitales, una de las que permite mejor colaboración y comunicación en tiempo real es Discord¹ (Figura 1).



Figura 1 - Discord.

Pese a las ventajas que ofrecen las comunidades digitales, uno de los grandes problemas en ellas surgen de la moderación de ellas, al tener varios usuarios ocultos en el anonimato de la web, la logística necesaria para mantener las comunidades como espacios de respeto y en armonía es compleja y en muchos casos requiere una alta colaboración entre moderadores y comunidad. Para facilitar esta tarea, la automatización es una herramienta ideal, pues permite entre otras cosas:

- **Automatización de tareas repetitivas:** Tales como dar la bienvenida a nuevos miembros, asignar roles y moderar contenido desde orígenes inapropiados pueden ser tareas que pueden gestionarse automáticamente con un programa correctamente diseñado para tal fin.
- **Mejorar la interacción:** Un programa puede responder a comandos específicos, permitiendo proporcionar información útil, y ejecutar acciones, haciendo que la experiencia de los participantes en la comunidad sea más dinámica y atractiva.
- **Moderación eficiente:** Un programa, a diferencia de un humano, puede monitorear en tiempo real el contenido, por lo que se pueden usar para identificar comportamientos inapropiados, lenguaje ofensivo o incluso acciones maliciosas.

¹ <https://discord.com/>

- **Personalización y escalabilidad:** Como estos programas están diseñados a medida, pueden adaptarse a las necesidades específicas de sus usuarios, implementando funciones útiles para los requisitos propios de cada comunidad.

A este tipo de programas se les conoce como *bots*, estos pueden ejecutar una serie de acciones complejas de forma automática según su programación. En este caso entenderemos un *bot* como un robot que no tiene un cuerpo físico, sino que su interacción funciona directamente a través de acciones en un programa. En esta oportunidad, el proyecto semestral que desarrollaremos se centrará en la creación de un *bot* para Discord orientado a mejorar la experiencia de usuarios a través de algunas funcionalidades que se detallarán más adelante en el presente documento.

BOT

Una de las herramientas que han hecho de Discord uno de los líderes en la gestión de comunidades en línea es la capacidad que ofrece a los usuarios de programar y desplegar *bots* que permiten la automatización de la moderación y otras tareas. Estos permiten principalmente reaccionar a ciertos comandos utilizados y apoyar al servidor enviando contenido, información, moderando, ayudando en la seguridad y otras acciones que pueden definirse directamente en su programación.

La mayoría de los *bots* de Discord funcionan a través de comandos de texto, en donde un mensaje tiene que poseer como mínimo 3 elementos básicos:

- **Prefijo o Prefix:** es el inicio del mensaje y se compone por una combinación de caracteres en particular. Si bien el *bot* siempre está leyendo los mensajes que entran, solo reaccionará a los que partan con estos caracteres. Por ejemplo, el símbolo de guion medio (-) es el *prefix* por defecto del *bot* YAGPDB². Por lo que este solo responderá a mensajes que inicien con este carácter.
- **Comando:** Luego del *prefix* en el mensaje, el *bot* debe interpretar que es lo que se le está solicitando. El comando es la acción misma que se le pide al *bot* que ejecute. Por ejemplo, el *bot* Pokecord³ tiene como *prefix* la combinación de la letra p y un signo de exclamación de cierre (p!), uno de sus comandos es “catch” e indica al *bot* que se desea atrapar a un pokémon. Por lo que el mensaje p!catch indica al *bot* Pokecord que se desea realizar esta acción.
- **Extras:** Luego del comando, en ocasiones el mensaje requiere parámetros adicionales para especificar opciones de la acción. Por ejemplo, si se enviará un saludo con el *bot*, puede ser necesario indicar el nombre del usuario al que se le realizará el saludo. Para ilustrar este funcionamiento consideremos el mensaje “p!catch squirtle”, en este caso se está pidiendo ejecutar una acción al *bot* Pokecord, identificado por el *prefix* p!, la cual es *catch*, es decir

² <https://yagpdb.xyz/>

³ <https://top.gg/bot/705016654341472327>

capturar un pokémon, y a su vez esta acción necesita la especificación del pokémon a capturar en este caso Squirtle, por ello, el extra *squirtle*.

Para poder determinar el comportamiento que un *bot* realizará, es necesario crear un programa que defina qué acciones ejecutará. Si bien esto puede realizarse en varios lenguajes de programación, como hemos trabajado todo el semestre con el lenguaje Python, utilizaremos el mismo en esta tarea. Para ello existe un módulo externo llamado *Discord.py*⁴ (Figura 4), el cual será de utilidad para la construcción de nuestra solución.

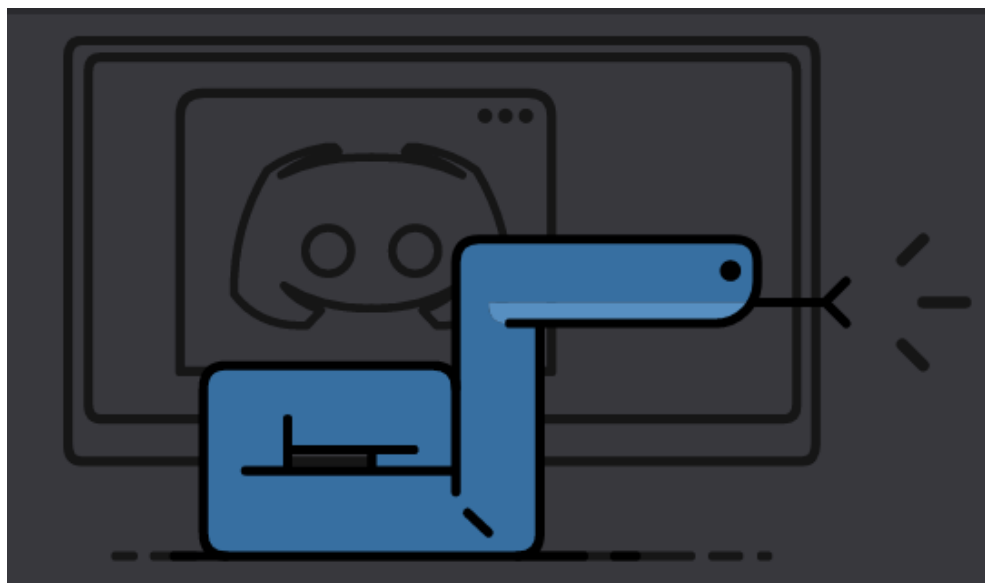


Figura 2 - Discord.py

Para ello, se les entregará un código base sencillo, el cual nos da una aproximación para entender las funcionalidades posibles del *bot*.

CREANDO EL BOT

Cada equipo de trabajo creará **un solo bot**, para ello, los miembros del equipo deberán crear una cuenta de Discord, si es que no tienen una, y luego, una vez iniciada la sesión acceder al siguiente enlace:

https://discord.com/login?redirect_to=%2Fdevelopers%2Fapplications

Se debería abrir la interfaz de la Figura 3, en ella deberán seleccionar “New Application” en el extremo superior derecho de la pantalla.

⁴ <https://discordpy.readthedocs.io/en/stable/>

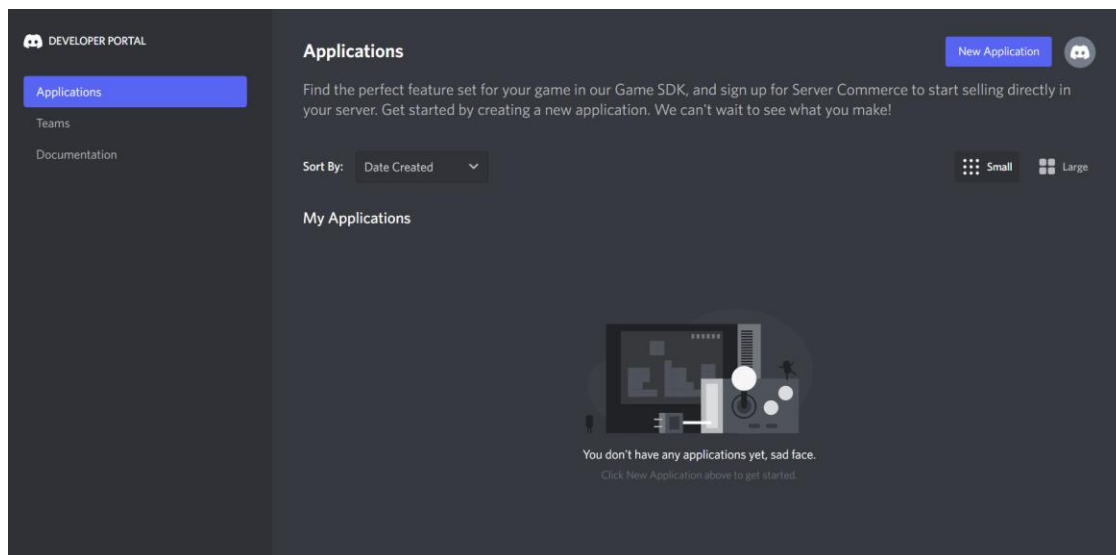


Figura 3 - Ventana de inicio de administración de aplicaciones de Discord.

Se les solicitará dar un nombre al *bot*, este puede ser modificado posteriormente, y la confirmación de la creación de este con el botón “Create” (Figura 4).

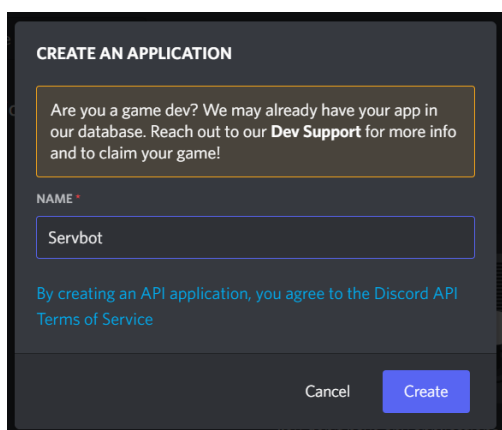
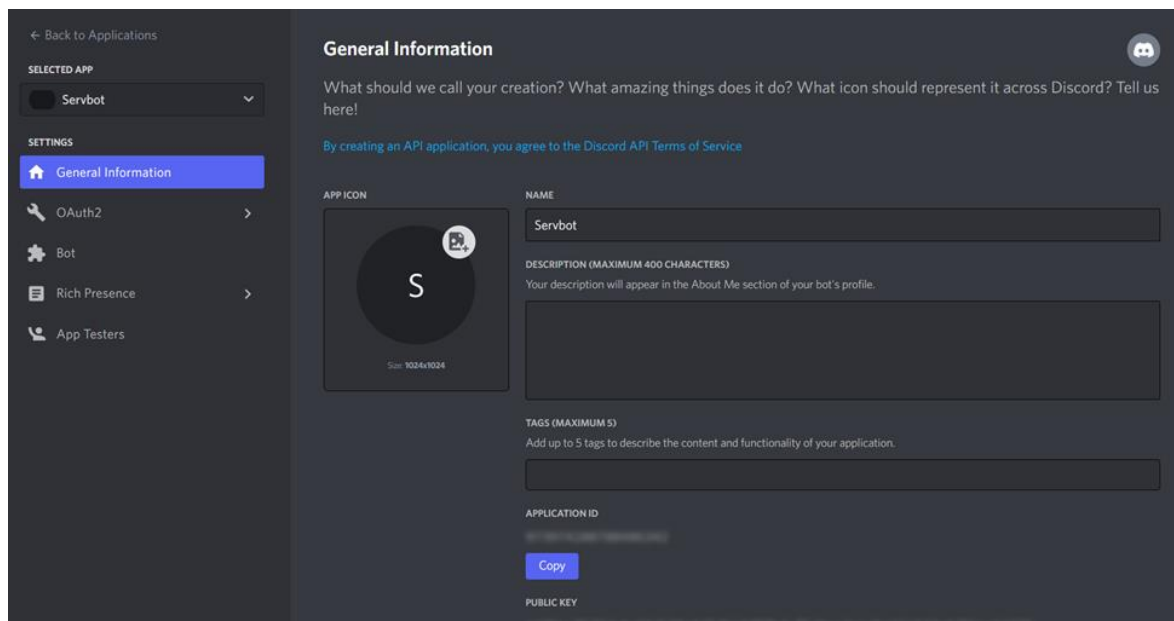


Figura 4 - Ventana de confirmación de creación de una aplicación.

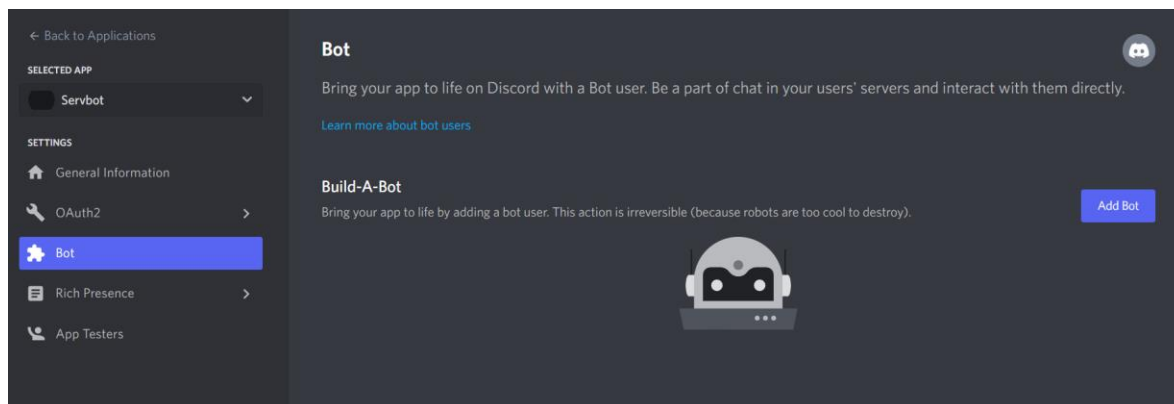
Aparecerá una nueva interfaz, en ella ya se puede configurar el *bot* (Figura 5). En este apartado es posible cambiar la imagen del *bot* y su nombre si lo desean. Cabe destacar que, como el contexto de este trabajo es académico, está completamente prohibido el uso de elementos ofensivos, improprios, contenido NSFW o cualquier otro elemento que no sea propios de una tarea universitaria. Esto no implica que esté prohibido que el equipo utilice nombres de fantasías o imágenes que expresen sus intereses personales (series, películas, deportes, videojuegos, etc.) mientras cumplan las indicaciones anteriores.



The screenshot shows the 'General Information' tab for a Discord application named 'Servbot'. The left sidebar contains a 'SELECTED APP' dropdown set to 'Servbot' and a 'SETTINGS' menu with options: 'General Information' (selected), 'OAuth2', 'Bot', 'Rich Presence', and 'App Testers'. The main content area includes a text prompt: 'What should we call your creation? What amazing things does it do? What icon should represent it across Discord? Tell us here!'. Below this is a link: 'By creating an API application, you agree to the Discord API Terms of Service'. The 'APP ICON' section shows a placeholder with the letter 'S' and a size of 1024x1024. The 'NAME' field contains 'Servbot'. The 'DESCRIPTION (MAXIMUM 400 CHARACTERS)' field has a placeholder text: 'Your description will appear in the About Me section of your bot's profile.' The 'TAGS (MAXIMUM 5)' field has a placeholder text: 'Add up to 5 tags to describe the content and functionality of your application.' The 'APPLICATION ID' field is visible with a 'Copy' button. The 'PUBLIC KEY' field is also visible.

Figura 5 - Ventana de configuración de la aplicación.

En el apartado de la derecha encontrarán la pestaña “bot” que permitirá configurar los elementos propios de este y ponerlo en producción, es decir, dejarlo disponible para que interactúe con la comunidad. En esta pestaña, lo primero es indicar que se quiere activar el *bot*, para esto hay que presionar en “Add bot” (Figura 6) y luego confirmar que se quiere dar visibilidad a este en la pantalla de confirmación (Figura 7).



The screenshot shows the 'Bot' tab for the same Discord application. The left sidebar is identical to Figure 5, but the 'Bot' option in the 'SETTINGS' menu is selected. The main content area has a heading 'Bot' and a subheading 'Build-A-Bot'. The text reads: 'Bring your app to life on Discord with a Bot user. Be a part of chat in your users' servers and interact with them directly.' Below this is a link: 'Learn more about bot users'. The 'Build-A-Bot' section has a text prompt: 'Bring your app to life by adding a bot user. This action is irreversible (because robots are too cool to destroy).' and an 'Add Bot' button. A robot icon is displayed at the bottom.

Figura 6 - Ventana de creación del bot.

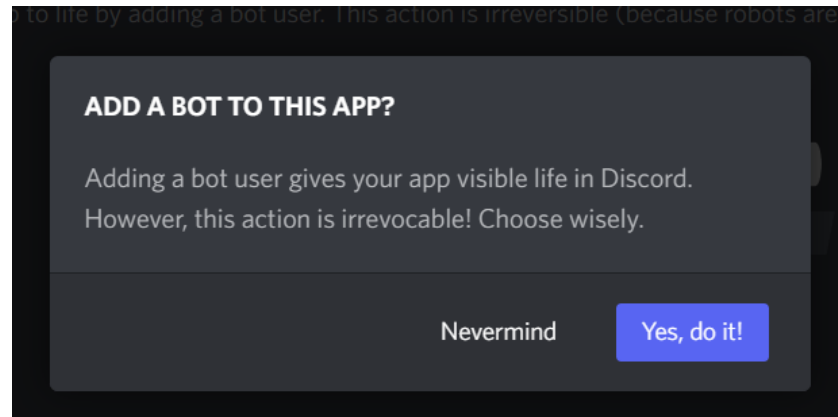


Figura 7 - Confirmación de creación del bot.

Una vez hecho esto, se agregarán un par de funcionalidades que no vienen activadas por defecto y que, si bien no se espera que sean utilizadas en este trabajo, es deseable tenerlas activas en caso de algún inconveniente. Para ello, nos desplazaremos a la sección “*Authorization Flow*” de la misma ventana.

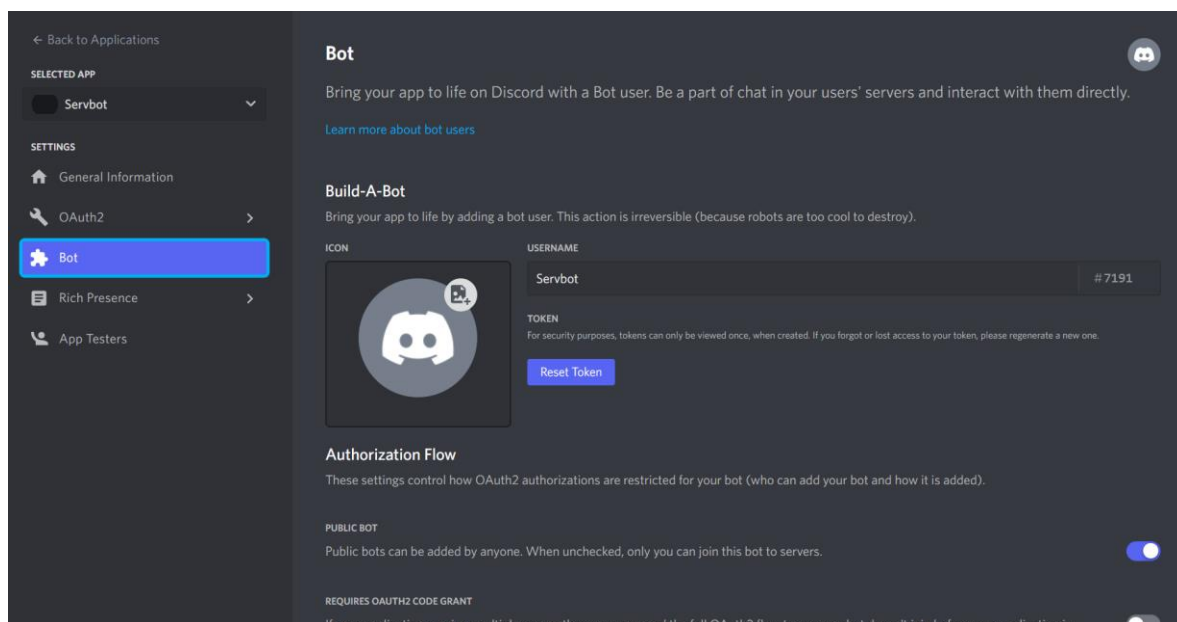


Figura 8 - Ventana de configuración del bot, una vez que este ha sido creado.

Active todas las opciones disponibles **excepto**: REQUIERES OAUTH2 CODE GRANT. Dejando los *checkbox* en color azul. Tras esto su “*Authorization Flow*” debería verse similar a lo que muestra la Figura 9.

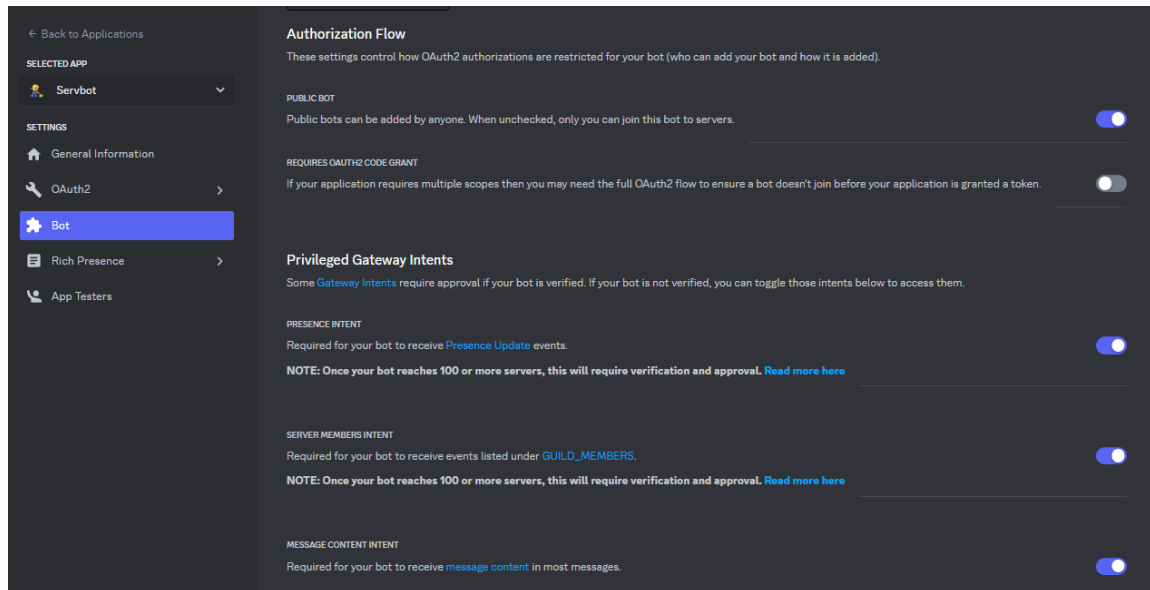


Figura 9 - "Authorization Flow" luego de activar los privilegios del bot.

CONECTANDO EL BOT

Una vez creado el *bot*, el siguiente paso es conectar el *bot* con el servidor y con el código en Python. Para ello se necesitan dos códigos de acceso: *ClientID* y *Token*.

- **Client ID:** Se puede obtener en la pestaña "General Information" (Figura 5) y es un número de cerca de 18 dígitos de largo que la interfaz lo presenta con el nombre de "Application ID" y es el código que permitirá crear la invitación para que el *bot* se una a un servidor.
- **Token:** Es una secuencia de caracteres (letras, símbolos y números) y se obtiene en la pestaña "Bot" (Figura 10), este solo es visible si se utiliza el botón "Reset Token" (Figura 11). Esta es una medida de seguridad para proteger el *token* ya que quien lo tenga podrá controlar al *bot*. En otras palabras, si el *bot* fuese un auto, el *token* es la llave que hace que alguien pueda entrar en él y utilizarlo. En caso de que usted o su equipo sospechen de una filtración del *token*, este debe ser reseedado y se debe reemplazar el *token* antiguo, por un nuevo *token* en todos los lugares en los cuáles se usó.

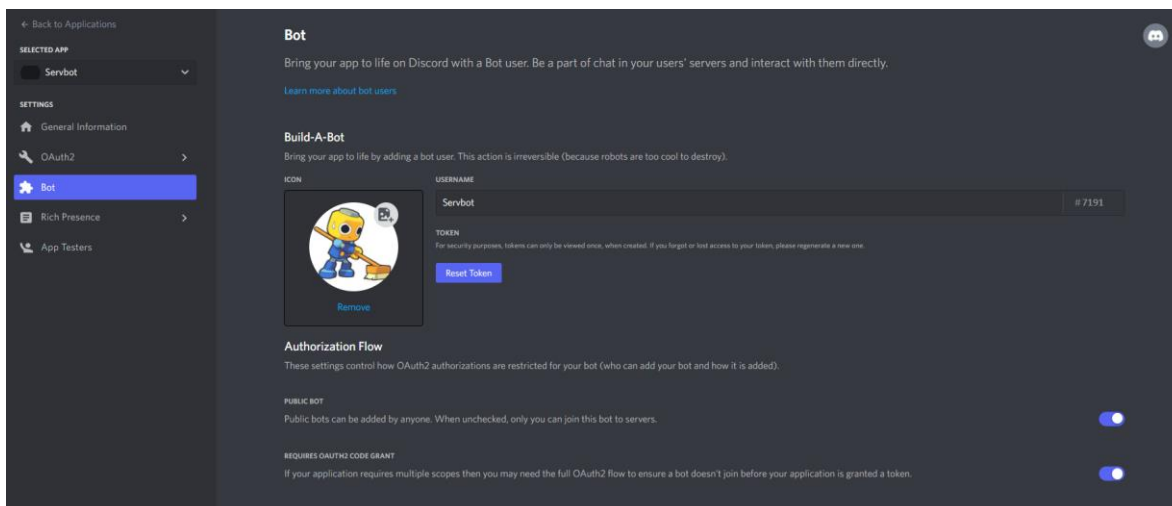


Figura 10 - Ventana de configuración del bot.

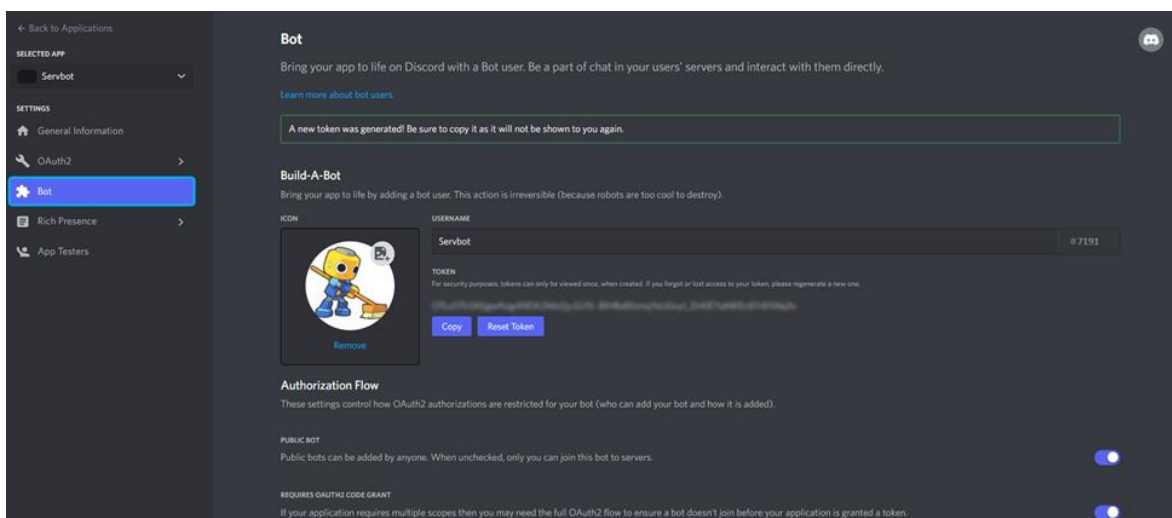


Figura 11 - Ventana de configuración del bot luego de resetear el token.

INVITAR AL BOT A UN SERVIDOR

Para poder invitar a un bot a un servidor es necesario utilizar el siguiente enlace:

https://discordapp.com/oauth2/authorize?&client_id=CLIENTID&scope=bot&permissions=8

Pero hay que cambiar la parte que dice CLIENTID por el ClientID que se obtuvo anteriormente (el número largo), quedando algo de la siguiente forma:

https://discordapp.com/oauth2/authorize?&client_id=123456789123456789&scope=bot&permissions=8

Este link debe ser enviado a un usuario de Discord que tenga privilegios de administrados en el servidor y podrá aceptar el ingreso del *bot* en el servidor. Durante el período de desarrollo, se recomienda a los estudiantes manejar un servidor propio para pruebas, posteriormente para la entrega, la coordinación probará los *bots* en un servidor en el cual el profesor tendrá privilegios de administrador.

CONECTANDO EL BOT CON PYTHON

Para conectar su código en Python con el *bot*, se le ha entregado un código base para trabajar. Este viene comentado, y se recomienda probarlo antes de comenzar a modificar su código. Este se compone de los siguientes elementos importantes.

En primer lugar, las instrucciones que importan el módulo `discord.py`. Este módulo no viene incluido con Python al momento de la instalación, por lo que es necesario instalarlo. La manera más sencilla es utilizando el comando **`pip install discord`** o **`pip3 install discord`** en la terminal. De la misma forma en que se instalan los módulos requeridos por el revisador.

```
from discord.ui import Button, View
from discord import app_commands
from discord.ext import commands
import discord
```

Luego, tenemos la identificación del *bot*. En las comillas hay que escribir el *token* del *bot* el cual será utilizado para realizar la conexión con el *bot*.

```
keyBot = ""
```

El siguiente apartado indica los privilegios que serán necesarios para crear al cliente, los cuales se activaron al momento de crear el *bot* en la sección anterior.

```
intents = discord.Intents.all()
```

Creación del *bot* con la información anterior, este objeto es el que indica quien se está conectando y realizará las acciones. En las comillas debe escribir el *prefix* que el *bot* utilizará.

```
bot = commands.Bot(command_prefix="", intents=intents)
```

A continuación, se mostrarán dos formas de interactuar, una es a través de comandos utilizados con el *prefix* y otra es reaccionando a cualquier mensaje enviado al servidor.

COMMANDS

```
@bot.command()
async def hi(ctx):
    await ctx.send("Hello Nurse...")
```

Esta es la forma de crear un nuevo comando, el nombre de la función “hi”, en azul, es el nombre del comando y se activará cada vez que el usuario utilice el *prefix* seguido de “hi”. La etiqueta **async** indica que la función se ejecuta de forma asíncrona del código. Esto se hace porque deseamos que pueda haber varias instancias de la misma función corriendo en paralelo, sin esperar que las anteriores terminen⁵. Las acciones que realizará el *bot* al ejecutarse el comando “hi” deben programarse dentro de esta función. Esta recibe como parámetro un objeto del tipo **Context** (ctx), el cual contiene información relevante respecto al mensaje, si bien puede explorar más este objeto, los elementos más significativos de este objeto son:

- **mensaje = ctx.message.content**: Este entrega el contenido del mensaje que se ha recibido en formato **string**. Este se le deja guardado en la variable local mensaje.
- **ctx.author**: Indica quien fue el que envió el mensaje, este elemento es otro objeto, en este caso del tipo **Member** que permite acceder a datos sobre el autor del mensaje. Si imprimimos “ctx.author” el programa nos dirá que existe un objeto de la clase *author* y nada más.
- **nombre_autor = ctx.author.name**: El atributo **name** de **Member** indica el nombre del autor del mensaje. El nombre del autor se le ha facilitado y dejado guardado en la variable local nombre_autor.
- **id_autor = ctx.author.id**: El atributo **id** de **Member** indica el identificador del autor del mensaje. Este número es único y no cambia, es la única forma de identificar inequívocamente a un miembro de la plataforma. El identificador del autor se le ha facilitado y dejado guardado en la variable local id_autor.
- **citar_autor = ctx.message.author.mention**: El atributo **mention** de **Member** entrega un **string** con el formato para poder citar a un usuario y etiquetarlo, esto gatillará una notificación para este usuario, por lo que se recomienda utilizarlo con discreción. El **string** resultante se le facilita guardado en la variable local citar_autor.

La sentencia `await ctx.send()` permite enviar un mensaje al servidor a través de un objeto **Context**. El mensaje será enviado al mismo servidor y canal del cual se recibió el mensaje. Para utilizarlo es necesario usar esta sentencia y como parámetro actual se debe utilizar un único **string** el cual será el que se envíe como mensaje a Discord, en este caso “Hello Nurse...”.

```
await ctx.send("Hello Nurse...")
```

Sentencia `await ctx.author.send()` permite enviar un mensaje privado al usuario `ctx.author`, este mensaje es un DM, por lo que solo ese usuario lo verá. Tenga en consideración que el mensaje podría fallar si el usuario bloquea al *bot*.

⁵ Los contenidos asociados a funciones asíncronas escapan del contexto de la asignatura y no es necesario profundizar más allá para realizar el proyecto. De todos modos, si desea entender este concepto más en detalle, se recomienda: <https://realpython.com/python-async-features/> (En inglés)

ON_MESSAGE

```
@bot.event  
async def on_message(message):
```

Esta es una función especial y será activada cada vez que se envíe un mensaje en los canales que el *bot* esté autorizado a visualizar (sin importar si hay un *prefix* o no, por lo que sea cuidadoso o el *bot* se descontrolará reaccionando a sus propios mensajes). El código debe estar dentro de esta función. La función recibe como parámetro un objeto **Message**, el cual permite acceder a mucha información sobre el mensaje y que se le hace entrega unos cuantos datos que necesita para desarrollar el proyecto:

mensaje = message.content: **message.content** entrega el contenido del mensaje que se ha recibido en formato **string**. Este se le deja guardado en la variable local **mensaje**.

message.author: indica quien fue el que envió el mensaje, es un objeto del tipo **Member** que permite acceder a muchos datos internos sobre el autor.

autor = message.author.name: El atributo **name** de **Member** indica el nombre del autor del mensaje. El nombre del autor se le ha facilitado y dejado guardado en la variable local **autor**.

autor_ID = message.author.id: El atributo **id** de **Member** indica un número entero que identifica a cada usuario de Discord. Este número es único y no cambia, es la única forma de identificar fehacientemente a un miembro de la plataforma. Este número se le facilita guardado en la variable local **autor_ID**.

citar_autor = message.author.mention: El atributo **mention** de **Member** entrega un **string** con el formato para poder citar a un usuario y etiquetarlo, esto gatillará una notificación para este usuario, por lo que se recomienda utilizarlo con discreción. El **string** resultante se le facilita guardado en la variable local **citar_autor**.

Sentencia `await message.channel.send()` permite enviar un mensaje al servidor. El mensaje será enviado al mismo servidor y canal del cual se recibió el mensaje. Para utilizarlo es necesario usar esta sentencia y como parámetro actual se debe utilizar un único **string** el cual será el que se envíe como mensaje a Discord.

```
await message.channel.send("hello "+ citar_autor)
```

La sentencia `await message.author.send()` permite enviar un mensaje privado al usuario `message.author` (creador del mensaje), este mensaje es un DM, por lo que solo ese usuario lo verá. Tenga en consideración que el mensaje podría fallar si el usuario bloquea al *bot*.

```
await message.author.send("ON Mensaje privado para "+  
message.author.name)
```

La sentencia `await bot.process_commands(message)` cierra la función `on_message` e indica que se debe continuar y pasar a revisar los comandos del *bot* con este mensaje para ver si el mensaje era realmente un comando.

Finalmente `bot.run(keyBot)` inicia el *bot* con el *token* que se especificó al inicio del código.

```
bot.run(keyBot)
```

El código que se desee agregar debe ser escrito dentro de la función `on_message()` o una función de comando como “hi” pero con el nombre que usted desee, analizado el mensaje recibido y tratando la información en él. Cabe destacar que si utiliza la sentencia `print`, solo se mostrará el resultado de este en el editor de Python, pero no el Discord, si quiere que algo se muestre en Discord debe utilizar las sentencia `await message.channel.send()` o `await ctx.send()` según corresponda, que puede considerarlos los `print` de discord.

PROYECTO

En esta oportunidad crearemos un *bot* de Discord orientado a una comunidad de estudiantes de algún curso masivo/carrera de ingeniería de la Universidad. Su equipo deberá escoger el curso/carrera y deberán levantar las temáticas de interés para la comunidad escogida.

A partir de esto, deberán diseñar algunas funcionalidades que deberá ejecutar el *bot* estas están divididas en tres tipos: Funciones generales, administrativas y de contenido.

FUNCIONES GENERALES

El *bot* debe ser capaz de saludar indicando el nombre del usuario quien saludó y además identificar el rol del usuario que lo saludó. Esto siempre y cuando el usuario utilice el comando para saludar del *bot*.

El *bot* debe tener un comando de ayuda que indique cómo utilizar las diferentes funciones del *bot*.

El *bot* debe ser capaz de enviar mensaje directo (DM) al usuario con información del servidor.

El *bot* debe ser capaz de alertar cuando alguien utilice improperios en algún mensaje.

El *bot* debe ser capaz de alertar cuando alguien envíe un mensaje con los siguientes links: 'linktr.ee/', 'twitch.tv/', 'patreon.com/', 'twitter.com/', 'x.com/', 'https://short.io/', 'onlyfans.com/', 't.co/', 'gumroad.com', 'sheer.com/', 'fans.ly/', 'tiktok.com/', 'https://discord.gg/', 'arsmate.com/' o 'https://cutt.ly'.

Entiéndase por alertar en este caso, etiquetar a el Team creador del *bot*.

ADMINISTRATIVAS

El *bot* debe ser capaz de consumir uno o más archivos .csv con información de algunas temáticas relevantes definidas por el grupo. Utilizando esto, se deben generar comandos para entregar información a los miembros del servidor sobre datos que puedan ser de su interés. Por ejemplo, crear comandos para preguntarle al *bot* sobre temas como:

- El temario de la PEP X de Cálculo 1.
- Dame el horario de la sección X-X de Fundamentos de Programación para Ingeniería.
- Donde están las salas EPF/ESID/DIINF, etc.
- A qué hora comienza el bloque X.
- ¿Quién es el jefe de carrera de Ingeniería Civil Eléctrica?

Se recomienda enfocar estas funcionalidades en el contexto específico. Por ejemplo, si su trabajo está orientado a estudiantes del curso de “Fundamentos de Programación para Ingeniería”, entonces su *bot* debe responder a consultar relevantes para este grupo.

FUNCIONES DE CONTENIDO

Como el objetivo es apoyar a alguna comunidad de Ingeniería de la Universidad, la idea es que el *bot* además de poder entregar información útil pueda tener interacciones que sean relevantes para los miembros de dicha comunidad. Es por ello que deberán implementar funcionalidades orientadas específicamente a problemas de la comunidad.

Por ejemplo, si el contexto fuera “estudiantes del curso de Fundamentos de Programación para Ingeniería”, podrían implementarse funcionalidades que respondan con alguna respuesta genérica a errores, tales como:

- Si el texto `' IndexError: list index out of range '` está presente en el mensaje, indicar: “El error `index out of range` se debe a que estás recorriendo más posiciones en un objeto iterable de las que tienes, esto puede deberse a que:
 - Estás recorriendo con un índice que llega más lejos de las posiciones existentes.
 - Estás eliminando elementos de la lista, acortando su tamaño mientras hay un proceso de recorrido.
 - Estás recorriendo una lista vacía.”

El desarrollo de estas funcionalidades puede ser definido por el grupo, durante el período de desarrollo del proyecto, el profesor irá verificando las funcionalidades y podrá dar retroalimentación respecto a la complejidad y viabilidad de dichas funcionalidades. El equipo docente puede definir si una funcionalidad no es relevante o es demasiado simple y vetarla del proceso de desarrollo. Del mismo modo, se considerará en la evaluación final la complejidad y la originalidad de cada funcionalidad.

ENTREGAS

El proyecto se dividirá en 3 entregas y vale el 40% de la calificación final del curso:

- **(2%) Entrega 1 (15/06):** Su equipo deberá completar la ficha del proyecto, indicando integrantes, carrera y proyecto escogido.
- **(8%) Entrega 2 (Semana 24/06):** Deberán presentar en no más de 8 minutos su avance al equipo docente de laboratorio
- **(30%) Entrega 3 (Semana 08/07):** Entrega final del proyecto. Esta consta tanto de la aplicación funcional y la documentación necesaria de esta. El sistema creado debe cumplir con todas las funcionalidades de forma completa.

Cada equipo estará compuesto por **máximo 5** estudiantes de la misma sección de laboratorio, que una vez inscrito **no podrá ser modificado**. Sin embargo, los equipos de trabajo **sólo deben identificar en las entregas a estudiantes que trabajaron**, si un estudiante no trabaja, se considera que no participó en la entrega y por lo tanto se calificará con nota mínima.

Si un estudiante desea cambiar de grupo, este deberá solicitar la autorización al profesor de laboratorio.
NO SE PERMITE TRABAJO INDIVIDUAL.

CONSIDERACIONES

Cómo toda actividad evaluada, el proyecto está sujeto al código de honor del curso. Ante cualquier caso de violaciones a las normas detalladas en él se sancionará con la **CALIFICACIÓN MÍNIMA EN EL PROMEDIO FINAL DEL PROYECTO** a todos los integrantes del (o los) grupo(s) involucrado(s)